

CHAPTER

1

THE CODE

The main part of this thesis is the implementation of a multi-orbital self-consistent ladder-D Γ A code with an emphasis on usability and minimal setup effort. Please note that there already exist two relatively modern implementations of a ladder-D Γ A code, one concerned with a one-shot multi-orbital calculation without asymptotics [Galler2019], and one with a λ -corrected approach for single bands using the “U-range” asymptotics described in Sec. ?? [Worm2023]. With the new code, we aim to combine both their functionalities, employing explicit multi-orbital asymptotics, a self-consistency loop with a prediction-based self-energy mixing scheme and an additional calculation of the superconducting eigenvalues through the multi-orbital Eliashberg equation. This allows for an improvement of the results obtained with the AbinitioD Γ A code from Ref. [Galler2019] for low-temperature data in order to study physically relevant multi-orbital materials such as the recently discovered bi-layer Nickelates La₃Ni₂O₇ in the near future. Both of these codes had an influence on the code described in this chapter and they also heavily influenced the design choices as discussed in Sec. 1.1 in more detail.

The code is designed to be both accessible and performant, targeting fast execution through efficient parallelization and extensive memory optimizations. It is written in Python and relies mainly on NumPy for numerical computations, making use of vectorized operations and optimized linear algebra routines. In addition, the code carefully exploits symmetries inherent to the physical problem to further reduce computational cost and memory usage. In this chapter, we present the implementation of the computational framework developed to achieve a fully converged self-energy through the self-consistency algorithm of the ladder-D Γ A equations and a subsequent calculation of the superconducting eigenvalues and gap functions for both singlet and triplet electron pairing.

Building upon the underlying theoretical foundations which were established already in the previous chapters, this numerical implementation aims to translate the formal expressions into a working algorithm. Special attention has been given to structuring the code in a modular and efficient manner, ensuring that it can be used not only for the ladder-D Γ A equations, but also for other algorithms that require multi-point vertex functions within the Matsubara frequency framework in condensed matter physics.

The package will be provided under the MIT license, making it freely available for use, modification, and distribution. This permissive license promotes collaboration and allows academics and developers to make changes on the code without placing restrictions. By publishing the implementation open-source, we hope to encourage accessibility, reproducibility, and community development.

In this chapter, we start with an overview of the overall structure and design choices of

the code, followed by a detailed discussion of its core components and algorithms. We then describe the typical workflow for using the code, including input handling, computational steps and output generation. Next, we show benchmark results against the already existing DGAPy code in the single-orbital case and perform consistency checks for the situation of fully disconnected bands and for an in orbital-space-rotated basis. Finally, we discuss potential improvements for future work.

1.1 Structure and design choices

As already mentioned above, there already exist two programs that allow the calculation of vertex functions and the self-energy through ladder-D Γ A: (i) AbinitioD Γ A [Galler2019, Galler2018, Galler2017], which is a one-shot multi-orbital calculation of the D Γ A self-energy and (ii) DGAPy [Worm2023], which is the one-band version of it with a λ -correction, where additional quantities are calculated, such as the superconducting eigenvalues and real-valued Green's functions. The former is written in Fortran, whereas the latter has been developed in Python. The purpose of the code developed in the course of this thesis is to provide the functionality of treating multi-orbital systems like AbinitioD Γ A, while being easily accessible and quick to set-up, similar to DGAPy. Therefore, we decided to write the program in Python. A main advantage of this programming language is its low entry-barrier and the easy set-up and code-execution. In theory, all that is needed is a Python environment with the necessary packages^[1], DMFT input files and the configuration file.

So far, DGAPy has successfully been used in several instances, for example in Refs. [DiCataldo2024, Worm2024], to only mention a few. Since it is already used among researchers, the transition from DGAPy to this code should be as simple as possible, hence the configuration file and some internal modules are designed similarly.

1.1.1 Parallelization via MPI

To lower the computation time, we employ the use of the mpi4py library in Python, which enables distributed computing across multiple processes in a straightforward and Pythonic manner. This allows the program to run inside a process pool, where objects can easily be sliced and passed to or gathered from other processes. While an ideal speed-up factor of n for n processes is theoretically possible, in practice, communication overhead and memory transfer costs typically limit the effective gain. Nevertheless, for large-scale computations involving high-dimensional vertex functions or large frequency boxes, this parallelization leads to significant reductions in runtime.

The central class handling parallelization is the `MpiDistributor`, which is initialized at the point where the explicitly non-local parts of the ladder-D Γ A algorithm begin. This class abstracts the communication logic and provides a consistent interface for scattering and gathering objects across processes. The current implementation slices the data along the transferred momentum $\mathbf{q}$, which is possible due to the independent parametric dependence of all equations up to the Schwinger-Dyson equation (??). Each MPI process is responsible for a specific set of $\mathbf{q}$ -points, allowing for highly parallel and independent evaluation of vertex contractions.

Internally, the slicing is guided by knowledge of the irreducible Brillouin zone, and the distributor automatically determines the mapping from global $\mathbf{q}$ -grid indices to process-local ones. This separation of concerns between parallelization logic and physics routines enhances maintainability

^[1]For a detailed list of requirements including package versions, see the file `requirements.txt` in the source code directory.

and flexibility of the code. Further MPI optimizations, such as asynchronous communications or load balancing across uneven grid sizes, could be explored in future versions.

1.1.2 Memory reduction through symmetry

The memory demands of a full ladder-D Γ A calculation can be substantial, especially for multi-orbital models and large frequency ranges. To mitigate this, the code incorporates several symmetry-based optimizations that significantly reduce the memory footprint and enable efficient scaling to larger systems.

First, the calculation of non-local quantities is performed exclusively within the irreducible Brillouin zone. Thanks to the crystal symmetries of common lattice structures (e.g. square or quasi-1D lattices), only a small subset of the full momentum grid needs to be explicitly computed. The code determines this set via symmetry operations specified in the `brillouin_zone` module, which ensures compatibility with arbitrary lattice symmetries defined in the configuration. For instance, on a $16 \times 16 \times 1$ grid, only 45 out of 256 $\mathbf{q}$ -points are needed for a square lattice, which directly reduces both computational and memory load by more than a factor of five and even higher for larger grid sizes.

To further save memory, we keep most of the objects in half of their bosonic Matsubara frequency range: Due to the symmetry $F_{1234}^{\omega\nu\nu'} = (F_{4321}^{(-\omega)(-\nu')(-\nu)})^* = (F_{1234}^{(-\omega)\nu\nu'})^*$ ^[2], we can restrict the objects to their positive bosonic Matsubara frequency region only. This saves half of the memory and thus a very large amount for the biggest objects, such as the auxiliary susceptibility or the full vertex.

These symmetry-based reductions are implemented in a transparent way for the user: the internal representation is compressed, but the high-level API provides access to the full data. This allows for both memory efficiency and usability, enabling the user to work with large systems that would otherwise be intractable on a single machine.

1.1.3 Object-oriented API

We tried to keep a very object-oriented approach when developing the toolbox for the program, as opposed to both DGApy and AbinitioD Γ A. Hence instead of directly working with NumPy arrays, we work with (Local)FourPoint, (Local)Interaction, SelfEnergy, GreensFunction and GapFunction classes. The reason being that in Python, it is very easy to overload operators, i.e. with so-called “dunder” methods, hence we implemented the most common operators “+/-/@/~”^[3], alongside multiplication and division with numbers, for these objects to help make the implementation of equations easier. This might not be an obvious improvement at first, however the underlying structure allows a very simple high-level handling of these objects, where most of the logic is performed in the background and only has to be coded once. We will discuss the explicit implementation of these dunder methods later.

Compared to DGApy, we stripped down the module responsible for the Matsubara frequency handling, since most of the features now can be called directly from the objects themselves. Instead of calling a method from the `matsubara_frequencies` module to cut an objects’ frequency range, it is now a method on the object itself, e.g. `object.cut_niv(10)`. Most of these methods are

^[2]The transformation from $-\nu'$, $-\nu$ to ν , ν' and the orbital swap are possible due to the symmetrized nature of the two-particle Green’s function extracted from DMFT and the inherited vertex functions.

^[3]Corresponding to addition, subtraction, multiplication and inversion.

only implemented once in a base class which all subtypes inherit from. These base classes already cover most of the objects' functionality, where the subclass only implements features unique to the specific object type.

All modules should be very self-explanatory, their names already provide a good description of what the module does and what its responsibilities are. For any details, we refer the reader to the documentation of the code inside the repository.

One benefit of the object-oriented approach is the now easy implementation of various equations. For example, writing an arbitrary equation with arbitrary (non-local) vertex functions A , B , C and D like

$$D_{1234}^{qv v'} = \frac{1}{2\beta} \sum_{v_1; ab} A_{12ab}^{qv v_1} (B_{ba34}^{\omega v_1} + C_{ba34}^{\omega v_1 v'}) \quad (1.1)$$

can be done with a call of $D = 1 / (2 * \text{beta}) * A.\text{matmul}(B.\text{add}(C))$ or the dunder methods for matrix multiplication “@” and addition “+”, i.e. $D = 1 / (2 * \text{beta}) * A @ (B + C)$, where the index contraction, frequency handling and summation will be done in the background. This eases readability in the code and reduces redundancy. Furthermore, it allows for very straightforward optimization of contractions and multiplications through this architecture. Furthermore, inverting a vertex function is also very easy now, all that needs to be called is the `obj.invert()` function (or with a “~” prefix, i.e. `~obj`) on any object and the code performs the inversion in the background using NumPy's vectorization and inversion by converting the object to compound indices. Furthermore, other operations, such as addition/subtraction in the form of $A.\text{add}(B)/A.\text{sub}(B)$ or just simply $A \pm B$ are possible and automatically handle different amounts of frequency dimensions, etc. There are a bunch of other convenience methods available, where we would like to refer the interested reader to the code and its documentation for more details.

1.1.4 Setup and execution

The code is very easy to set-up as it is available as an installable Python package, called `scdga`. After installing the necessary packages denoted in the `requirements.txt` file to the Python environment, one simply has to run “`pip install -e .`” from the repository directory in the terminal to install the package in editable mode. This allows the user to now access any module and class inside the `scdga` package. The main entry point to the program is the file `dga_main.py` file, which can be started with either “`python dga_main.py`” for single-core execution (mostly used for testing purposes) or “`mpiexec -np <n_proc> python dga_main.py`” for multi-core processing. There are two additional command line parameters available to append to the execution:

- “-p”: With “-p <path>” one can specify the path to the configuration file, which contains all run-specific parameters. This is useful if one wants to store multiple configuration files in different directories. If this parameter is not set, the path defaults to the location of the repository directory.
- “-c”: With “-c <config name>” one can specify the name of the configuration file one wants to load. This defaults to `dga_config.yaml`.

As an example, the following shell command runs the code using 8 MPI processes and loads the configuration file `my_config.yaml` from the path `/configs/`:

“`mpiexec -np 8 python dga_main.py -p /configs/ -c my_config.yaml`”.

Additionally, we feature extensive logging: every important step in the calculation will be logged. If it is started from a terminal, the logging will be done to the standard output. If the code is executed on a slurm-based cluster, one will find the logs in the job output file. The reason we employ a lot of logging is the ease of finding errors that might occur during a calculation.

1.1.5 Configuration

The configuration file is split into small blocks of configuration sections, which will be explained in detail in the following. For those entries where only a single data type is expected, the code will try to parse the input to this specific type. If the parsing was unsuccessful, a warning is logged and a default value for this variable is used.

The first block of the configuration is concerned with the number of Matsubara frequencies one wants to use for the calculation:

```
box_sizes:
  niw_core: 50 # int, default: -1
  niv_core: 30 # int, default: -1
  niv_shell: 20 # int, default: 0
```

The “core” region defines the frequency box used for explicitly solving the Bethe-Salpeter and Schwinger-Dyson equations, while the “shell” region sets the size of the asymptotic tails needed for vertex reconstruction through the method described in Sec. ???. It is possible to set the core frequencies to “-1”, which means that the number of Matsubara frequencies from the DMFT calculation will be taken.

The next section is concerned with the Hamiltonian of the system, which currently is a little bit complex, since we wanted to adapt the existing configuration file from DGAPy and added new features. It defines the lattice symmetries, the momentum grid sizes, the non-interacting and interacting part of the Hamiltonian:

```
lattice:
  symmetries: "two_dimensional_square"
  type: "from_wannier90" # string, default: "two_dimensional_square"
  hr_input: "/path/to/file" # string | list[float], default: None
  interaction_type: "one_band_from_dmft"
  # string, default: "one_band_from_dmft"
  interaction_input: "/path/to/file"
  # string, default: ""
  nk: [16, 16, 1] # list[int], default: [16, 16, 1]
  nq: [16, 16, 1] # list[int], default: [16, 16, 1]
```

Here, we have a bunch of settings available that can be combined in different ways. First, we can define the lattice symmetry, where one has to enter a set of predefined symmetry sets. It is possible to extend the symmetries the code supports by adding the corresponding symmetry operations and symmetry sets in the code. The available symmetry sets are `two_dimensional_square`, `quasi_one_dimensional_square`, `simultaneous_x_y_inversion` and `quasi_two_dimensional_square_symmetries`. Next is the type of input to the non-interacting part of the Hamiltonian we require. It can either be `from_wannier90`, `from_wannierHK` or `t_tp_tpp` (only for single-band input). The first input type is used if the subsequent field `hr_input` references to a file, where the hopping elements are written in real space, whereas `from_wannierHK` is used if the Hamiltonian in Fourier space is

available as a file^[4]. The last option is only used for single-band input, where the band distribution is that of a t , t' and t'' hopping model only. If this type is specified, then a list of three float values in `hr_input` is expected.

Since this code is capable of performing multi-orbital calculations with some non-local interactions, we require the definition of a interaction type and an input. The available interaction types are `one_band_from_dmft`, `kanamori_from_dmft` and `custom`. If `one_band_from_dmft` is specified, we expect a single-band input, where the interaction strength is taken from DMFT. Similar to that, the `kanamori_from_dmft` option takes the interaction values from DMFT and constructs the correct interaction matrix with the Kanamori-specific entries. In both cases, the field `interaction_input` will not be read. Lastly, if one specifies `custom` as the interaction type, we then expect a path to a file, which is structured similarly to a real-space Hamiltonian file but with four orbital entries instead of just two. It is possible to include non-local interactions there. Furthermore, it is possible to set the sizes of both the $\mathbf{k}$ - and $\mathbf{q}$ -grid in the fields `nk` and `nq`, respectively. It is possible to not set `nq`, where `nk` will be used for the $\mathbf{q}$ -grid. Note, that the execution of the Eliashberg equation is only possible if both the $\mathbf{k}$ - and the $\mathbf{q}$ -grid are of the same size.

The next configuration section is concerned with the self-consistency calculation and defines the convergence behavior and mixing strategy of the self-consistency loop:

```
self_consistency:
    max_iter: 20                # int, default: 20
    save_iter: True             # bool, default: True
    epsilon: 1e-6               # float, default: 1e-4
    mixing: 0.3                 # float, default: 0.2
    mixing_strategy: "pulay"    # string, default: "linear"
    mixing_history_length: 2     # int, default: 3
    previous_sc_path: "path/to/files" # string, default: "./"
```

The field `max_iter` is to set an upper boundary on the number of iterations during the self-consistency cycle. If the self-energy does not converge within `max_iter` iterations, the iteration will stop. When setting `save_iter` to `True`, the code will save the non-local $D\Gamma A$ self-energy for each iteration. The next parameter, `epsilon`, is set to determine the absolute value difference allowed for the last two calculated sigmas to be considered converged. The convergence will be tested on the full $\mathbf{k}$ -grid and first $\nu_{n,\max} = \left\lceil \frac{n_{\text{iv_core}}}{5} \right\rceil$ frequencies^[5], i.e. the self-energy is considered converged when for all values $\mathbf{k}$ and $\nu_n \in \{1, \dots, \nu_{n,\max}\}$ the expression $|\Sigma_l(\mathbf{k}, \nu_n) - \Sigma_{l-1}(\mathbf{k}, \nu_n)| \leq \text{epsilon}$ holds, where l denotes the iteration number.

Next, we can specify the mixing parameter that will be used in the mixing scheme and is a floating point number between $[0, 1]$. We can furthermore specify the mixing scheme in the next parameter, called `mixing_strategy`. Here, we can choose from either `linear` or `pulay` mixing, where if `pulay` is specified, it takes `mixing_history_length` number of previous self-energy results to construct a prediction. If the number of iterations is less than the history length, we will use linear mixing for these iterations and switch to Pulay mixing afterwards. The last parameter, `previous_sc_path` allows us to specify the path to a previous self-consistency iteration folder, which might not have been converged or which needs to be converged further or with different parameters. The program then takes the last iterations of the already performed calculation and uses these as a starting point

^[4]Note that for the former, it is possible to use arbitrary $\mathbf{k}$ -grid sizes, since the Fourier transform will be performed in the code, whereas for the latter one the correct grid size has to be entered.

^[5]This is just an estimate, since for low frequencies the self-energy does not display significant statistical noise carried over from DMFT.

for the next calculation. If the number of iterations from the previous calculation is equal or larger than `mixing_history_length` and `pulay` is specified, then Pulay mixing will be applied, otherwise we mix linearly.

Given the improved speed and scalability of this code compared to DGApy, we have also implemented two λ -correction schemes which only work for single-band cases and can be configured with the following short section:

```
lambda_correction:
    perform_lambda_correction: True # bool, default: False
    type: "spch"                  # string, default: "spch"
```

Here, setting `perform_lambda_correction` to `True` will perform a λ -correction if the input is a single-band model. Otherwise, an error is raised and the code exits. Notice that if one calculates the one-shot λ -corrected self-energy and Eliashberg equation, the variables `max_iter` and `mixing` in the `self_consistency` section will be set to “1” during execution. The other parameter is the type of λ -correction which will be performed. Setting this to `spch` will renormalize both the density and magnetic susceptibilities, whereas setting it to `sp` will only renormalize the magnetic susceptibility.

The next section is a very important one, since it is concerned with the location of the DMFT files and mainly defines their location and file names:

```
dmft_input:
    type: "w2dyn"                # string, default: "w2dyn"
    input_path: "/path/to/files" # string, default: "/"
    fname_1p: "filename"         # string, default: "1p-data.hdf5"
    fname_2p: "filename"         # string, default: "g4iw_sym.hdf5"
    do_sym_v_vp: True            # bool, default: True
```

Currently, the `type` argument only supports `w2dyn` as input type, but this could be extended in the future to also support input files from other sources with other file structures. Next, `input_path` needs to be set to the folder that contains the output data from `w2dynamics`. We require two files, one containing the DMFT self-energy and impurity Green’s function and one containing two-particle quantities (i.e. the two-particle Green’s function in both density and magnetic channels); their filenames can be set in `fname_1p` and `fname_2p`, respectively. Notice that `w2dynamics` outputs a large `Vertex.hdf5`-file, where we have to extract the relevant worm components beforehand. This can be done with the script `symmetrize.py`, which is also part of the repository^[6]. This will automatically extract all relevant worm components of the two-particle Green’s function and writes it to a separate file containing the density and magnetic contributions which is required for `fname_2p`, while the file for `fname_1p` can be directly taken from the one-particle DMFT calculation. A straightforward extraction of the two-particle Green’s function from the `Vertex.hdf5` file is not possible. Lastly, we have the option to symmetrize the two-particle Green’s function in v and v' before processing them further. This might be necessary sometimes to ensure numerical symmetry of the vertex functions.

The second to last configuration section we will cover here is the output section which, as the name already suggests, covers the program’s output settings, such as the output path and plotting

^[6]In the corresponding repository of Ref. [Galler2019] a file with a similar name is included, however the functionality between both versions is slightly different.

instructions:

```
output:
  output_path: "/path"           # string, default: "./"
  do_plotting: True              # bool, default: "True"
  plotting_subfolder_name: "Plots" # string, default: "Plots"
  save_quantities: True          # bool, default: True
```

We are able to choose the path where the output of the program is saved to. If this field is chosen empty, the output will be put into a sub-folder inside the DMFT input folder. If we specify `do_plotting`, then we plot a selective set of quantities and save them to the sub-folder specified in `plotting_subfolder_name`. We then have the possibility to additionally save almost all quantities that are calculated during the main program as `.npz` files, except for the pairing vertex and the full ladder- $D\Gamma A$ vertex. These are separately handled in the Eliashberg section, since they are only calculated if one wants to perform the calculation of the superconducting eigenvalue and gap function.

Finally, the last configuration section is concerned with the Eliashberg equation and is configuring the eigenvalue and gap function-finding through a Lanczos method:

```
eliashberg:
  perform_eliashberg: True      # bool, default: False
  save_pairing_vertex: False    # bool, default: False
  save_fq: False                # bool, default: False
  n_eig: 4                      # int, default: 4
  epsilon: 1e-9                 # float, default: 1e-6
  symmetry: "random"            # string, default: "random"
  include_local_part: True      # bool, default: True
  subfolder_name: "Eliashberg" # string, default: "Eliashberg"
```

Here, the first setting is to let the program know if one wants to perform the Eliashberg equation to obtain the superconducting singlet and triplet eigenvalues and gap functions, respectively. If this setting is set to “False”, then the program will exit after the self-energy has been calculated. The next two settings allow to save the (non-symmetrized) pairing vertex $\tilde{\Gamma}^{q=k-k';\nu\nu'}$ and the full ladder vertex $F^{q\nu\nu'}$ to a file. These are set to “False” per default, as these objects can be quite large, especially the full vertex, which can reach up to hundreds of gigabytes. Both will be saved for the irreducible Brillouin zone only to save storage space. The following three settings, `n_eig`, `epsilon` and `symmetry` are concerned with the power iteration that is performed to solve the Eliashberg equation. `n_eig` allows the user to specify the number of largest eigenvalues and corresponding eigenvectors to be calculated using a Lanczos method. `epsilon` determines the target precision of the power iteration which is performed and `symmetry` determines the starting vector. There are a couple of options available for `symmetry` that may help speeding up the power iteration convergence if one knows the pairing symmetry of the gap function a priori: `random`, `p-wave-x`, `p-wave-y` and `d-wave`. In almost all cases it should suffice to pick a random starting vector for the power iteration.

In principle one also needs the *purely local* full and reducible vertices for the pairing vertex, however these can be omitted if one expects p- or d-wave symmetry of the gap function, where the local vertex functions do not contribute to the eigenvalue and gap function due to symmetry. Setting `include_local_part` to “False” does exactly that. Not calculating the local part allows for a slightly larger frequency box and a better estimation of the eigenvalue and gap function for d-wave symmetry cases.

The last setting is the sub-folder name where the Eliashberg output will be saved to.

1.1.6 Input files

After the vertex calculation has been performed with w2dynamics [Wallerberger2019], we have to extract a couple of key quantities, such as the local self-energy, Green's function, interaction and two-particle Green's functions from the output files of w2dynamics. There is a script located in the repository, called `symmetrize.py`, which allows for the straightforward extraction of the two-particle Green's function from the vertex file of w2dynamics. A simple execution (with “python `symmetrize.py`”) of this script yields a new file, where the density and magnetic components of $G_{r,1234}^{\omega vv'}$ are written to. This is handy, since the vertex file from w2dynamics is usually very large and it can be deleted after the symmetrization process. Then one is left with the `1p-data.hdf5` and `g4iw_sym.hdf5` files, which contain the one-particle and two-particle (symmetrized) quantities. These are, apart from a file containing the real-space (e.g. `wannier_hr.dat`) or Fourier-space (e.g. `wannier.hk`) Hamiltonian and the configuration file, the only input files needed for the execution of the D Γ A algorithm. Notice that no file name is fixed and they can be specified in the corresponding `dga_config.yaml` file.

Since we only care about the density and magnetic spin combinations for the two-particle vertex functions and can disregard any spin combination where there is an odd number of $\uparrow$ and $\downarrow$ or any of the form $\uparrow\downarrow\uparrow\downarrow$, we only have to sample a reduced set of worm components. Out of the possible $16 \times n_o^4$ (total number of spin-orbit combinations), there only exist $6 \times n_o^4$ non-zero worm components. This reduces the computational time of the QMC run by more than a factor of 2.5. The reduced worm component list can be retrieved by a call to the function `get_worm_components` in the file `symmetrize.py`. This means that for a single band we only have to sample 6/16, for two bands 96/256 and for three bands 486/1296 components. Since in principle for SU(2) symmetry the number of spin components can be reduced to two^[7], the number of worm components can be further reduced to $2 \times n_o^4$ at the cost of stochastic noise which is lowered by combining more spin combinations.

1.2 Numerical effort

In the following, we will briefly analyze the numerical scaling of the implemented D Γ A algorithm, which is in principle the same as in [Galler2019]: For low temperatures, the calculation of the local vertex functions through the CT-HYB QMC solver is very often the most time-consuming part of a D Γ A calculation. The numerical cost of evaluating the vertex scales as $\mathcal{O}(\beta^5 n_o^4)$ with a considerable prefactor due to the Monte Carlo sampling of w2dynamics. Here, n_o is the number of orbitals (bands). This scaling arises because the local vertex contains $\mathcal{O}(n_\omega n_v^2 n_o^4)$ components^[8], while updating the CT-HYB hybridization matrix itself requires $\mathcal{O}(\beta^2)$ operations: the average expansion order and the matrix dimension both grow linearly with β .

To mitigate the cost, the asymptotic high-frequency form of the irreducible vertex, see ??, can be calculated with only two frequencies and therefore scales as $\mathcal{O}(\beta^4 n_o^4)$, allowing for a smaller frequency-box size in the QMC calculation. This makes calculations near superconducting transition temperatures feasible.

In the new code, the calculations are parallelized over the system's $\mathbf{q}$ -grid and are diagonal in the bosonic Matsubara frequency ω , yielding a numerical effort that scales with $\mathcal{O}(n_q n_\omega)$. For each $\mathbf{q}$ and ω entry, the dominant cost stems from operations using vectors and matrices whose dimensions

^[7] $\{\uparrow\uparrow\uparrow\uparrow, \uparrow\uparrow\downarrow\downarrow\}$ instead of $\{\uparrow\uparrow\uparrow\uparrow, \downarrow\downarrow\downarrow\downarrow, \uparrow\uparrow\downarrow\downarrow, \downarrow\downarrow\uparrow\uparrow, \uparrow\downarrow\uparrow\downarrow, \downarrow\uparrow\downarrow\uparrow\}$

^[8] In general, we assume that the number of Matsubara frequencies scales with β , i.e. $n_\omega \sim n_v \sim \beta$.

are $N = n_v n_o^2$. Therefore, a matrix-matrix multiplication or straightforward inversion of a single matrix scales as $\mathcal{O}(N^3)$ ^[9], whereas a matrix-vector multiplication scales as $\mathcal{O}(N^2)$. Consequently, the overall computational cost for these operations behaves as $\mathcal{O}(n_q n_\omega n_v^3 n_o^6)$ and $\mathcal{O}(n_q n_\omega n_v^2 n_o^4)$, respectively.

The evaluation of the self-energy through the Schwinger-Dyson equation has a different scaling, $\mathcal{O}(n_k n_q n_\omega n_v n_o^6)$ and only becomes dominant for very dense momentum grids.

Furthermore, the whole algorithm scales linearly with the number of self-consistency iterations, adding an additional factor of n_{iter} to the full one-shot D Γ A cycle.

1.3 Benchmarks and internal consistency checks

In order to check if the program's output is correct, we created a few simple test cases to benchmark the algorithm and check our code for internal consistency:

- (i) First, we verify that the new code reproduces the same results as DGApY. While this comparison is limited to single-band models, it remains a crucial benchmark. Since the new implementation also supports λ -correction for single-band data, we compare both codes across two λ -correction schemes. Further details are provided in Sec. 1.3.1.
- (ii) Next, we assess the correctness of the multi-orbital implementation. To this end, we constructed a artificial two-orbital dataset in which both bands are identical, fully disconnected and inter-orbital interactions are absent. Using the same parameters as in the previous test, we compare the results for each orbital with those from the single-band case. Aside from statistical noise, we expect the results to be identical. This consistency check is discussed in more detail in Sec. 1.3.2.
- (iii) Finally, we test the invariance of the results under basis transformations in orbital space; here we take orbital-space rotations. Starting from the previous test case, we rotate the DMFT input data — including the local one- and two-particle Green's functions, the DMFT self-energy, the Hamiltonian and the interaction matrix — in orbital space. After performing the calculation, we apply the inverse rotation to the output and compare the result with the unrotated case. As detailed in Sec. 1.3.3, the two results have to coincide within numerical precision.

For the single-band input, we used $\beta = 12.5/t$, a filling of $n = 0.85$ and tight-binding parameters representative of the Ni- $d_{x^2-y^2}$ orbital in $\text{Nd}_{1-x}\text{Sr}_x\text{NiO}_2$ and related compounds: $t = 1$, $t' = -0.25t$ and $t'' = 0.12t$, with an interaction strength of $U = 8t$. Analogous parameters were used for the artificial two-band input, where the total filling was set to $n = 1.7$, corresponding to 0.85 electrons per band. The Hamiltonian was constructed to be orbital-diagonal, using identical hopping amplitudes and interaction strengths for both bands. All input data were generated using the CT-HYB quantum Monte Carlo solver w2dynamics, run on the Vienna Scientific Cluster.

^[9]In our implementation, we use the inversion provided by the Python package NumPy, which employs the LAPACK functions cgetrf/cgetri or zgetrf/zgetri, depending on the precision required. These algorithms scale as $\mathcal{O}(N^3)$, whereas other algorithms with a lower complexity exist, such as Strassen-like or $N^{2.5}$ -type inversions. However, these are typically only available in specialized libraries and are rarely used for dense numerical inversion in scientific computing because of stability and memory tradeoffs.

1.3.1 Benchmark against DGApy for the single-orbital model

We performed a full one-shot D Γ A calculation using both DGApy and our new implementation, employing identical input files and parameters. Across all computed quantities, our implementation reproduced the results of the D Γ A algorithm with floating-point accuracy. This agreement held for both the basic one-shot calculation without λ -corrections and for cases with λ -corrections applied — whether in the spin channel alone or in both spin and charge channels.

We discovered a tiny subtlety in the implementation of DGApy. It uses a different double-counting correction, which reads for the single-band case

$$\Sigma_{\text{dc}}^k = \frac{1}{\beta} \sum_{\mathbf{q}\mathbf{v}'} u \chi_0^{\mathbf{q}\mathbf{v}'} F_{\mathbf{m}}^{\omega\mathbf{v}'\mathbf{v}} G^{\mathbf{k}-\mathbf{q}}, \quad (1.2)$$

compared to the expression containing a combination of the full vertex in the density and magnetic channel, see our definition of Eq. (??). This does not add a significant numerical differences here, but it should be noted nevertheless as it becomes more relevant for lower temperatures. The usage of Eq. (1.2) in DGApy is for the special case of single-band data, where one can show, that

$$\sum_{\mathbf{q}\mathbf{v}'} \chi_0^{\mathbf{q}\mathbf{v}'} [F_{\text{d}}^{\omega\mathbf{v}'\mathbf{v}} + F_{\text{m}}^{\omega\mathbf{v}'\mathbf{v}}] G^{\mathbf{k}-\mathbf{q}} \propto \sum_{\mathbf{q}\mathbf{v}'} \chi_0^{\mathbf{q}\mathbf{v}'} F_{\uparrow\uparrow}^{\omega\mathbf{v}'\mathbf{v}} G^{\mathbf{k}-\mathbf{q}} = 0. \quad (1.3)$$

We carried out the single-band calculation with Eq. (1.2) and Eq. (??) and compared them to the results of DGApy separately, both of which can be seen in Fig. 1.1 and Fig. 1.2, respectively. These calculation were done with a $\mathbf{k}$ -grid of $64 \times 64 \times 1$, $n_{\omega} = 60$, $n_{\mathbf{v};\text{core}} = 40$ and $n_{\mathbf{v};\text{asympt}} = 20$ without any λ -correction. The absolute differences in the self-energies between the two implementations did not exceed $\mathcal{O}(10^{-11})$, and the five largest eigenvalues of the pairing vertex in the Eliashberg equation differed by at most $\mathcal{O}(10^{-6})$ for the double-counting kernel of Eq. (1.2). This level of agreement was consistently achieved across different $\mathbf{k}$ - and $\mathbf{q}$ -grids and varying numbers of Matsubara frequencies n_{ω} and $n_{\mathbf{v}}$.

For the double-counting kernel of Eq. (??) we found an agreement of $\mathcal{O}(10^{-3})$, where the largest superconducting eigenvalues differed by $\mathcal{O}(10^{-4})$.

A comparison between the *local* irreducible vertices for both calculations and both density and magnetic spin combinations can be seen in Fig. 1.3 and Fig. 1.4, respectively.

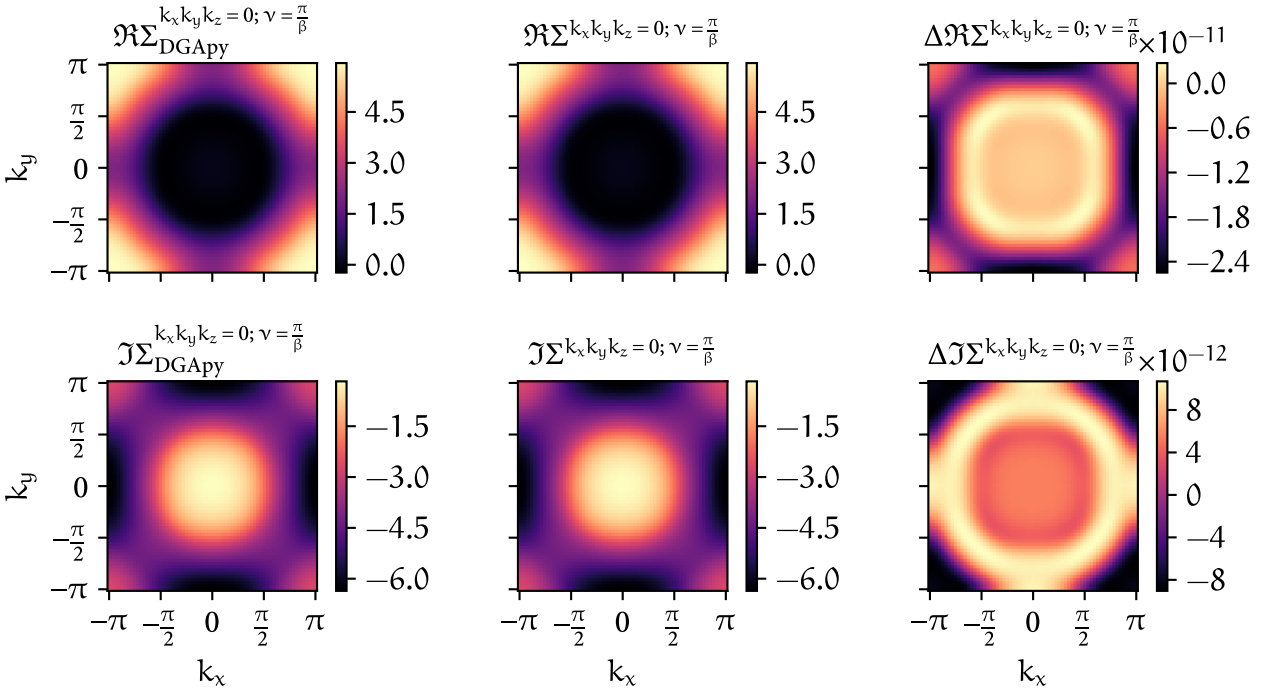


FIGURE 1.1 – Difference between the one-band self-energy of DGAPy (first column) and the current implementation with the double-counting kernel of Eq. (1.2) (second column) for $k_z = 0$ and $v = 0$. The difference between both, i.e. $\Delta\Sigma = \Sigma_{\text{DGAPy}} - \Sigma$, can be seen in the last column. The rows indicate the real- and imaginary part of all self-energies, respectively.

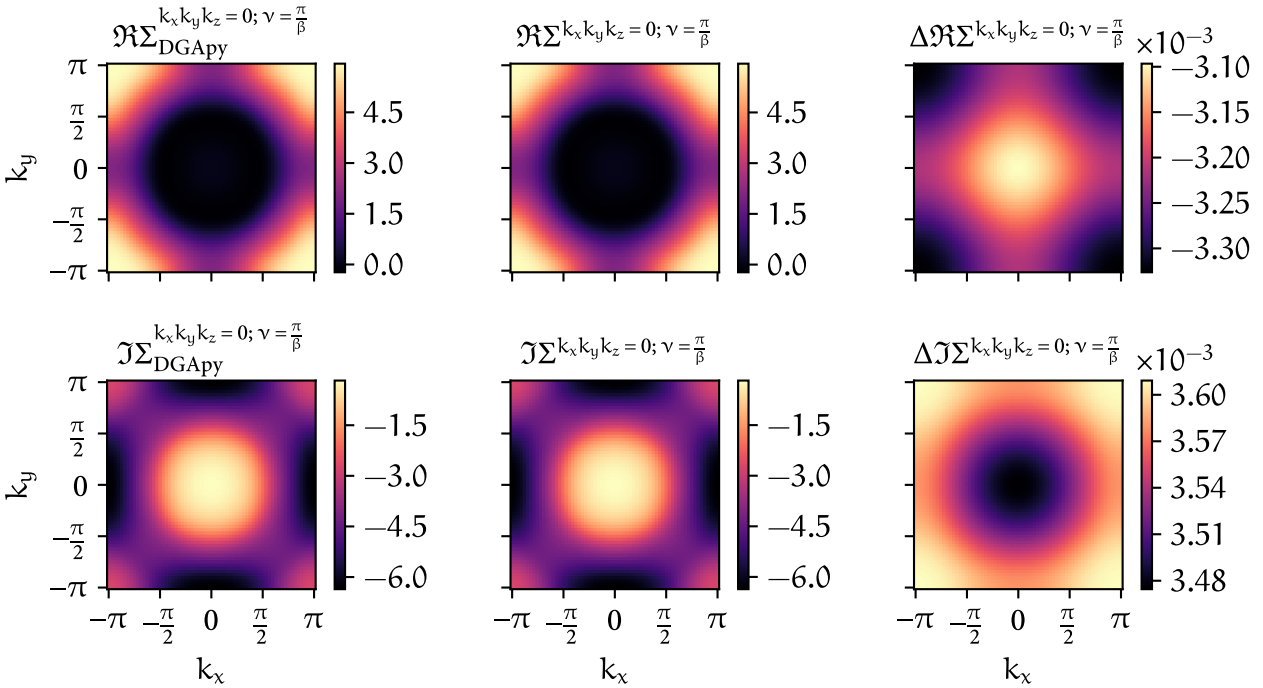


FIGURE 1.2 – Analogous to Fig. 1.1, but with the double-counting kernel of Eq. (??).

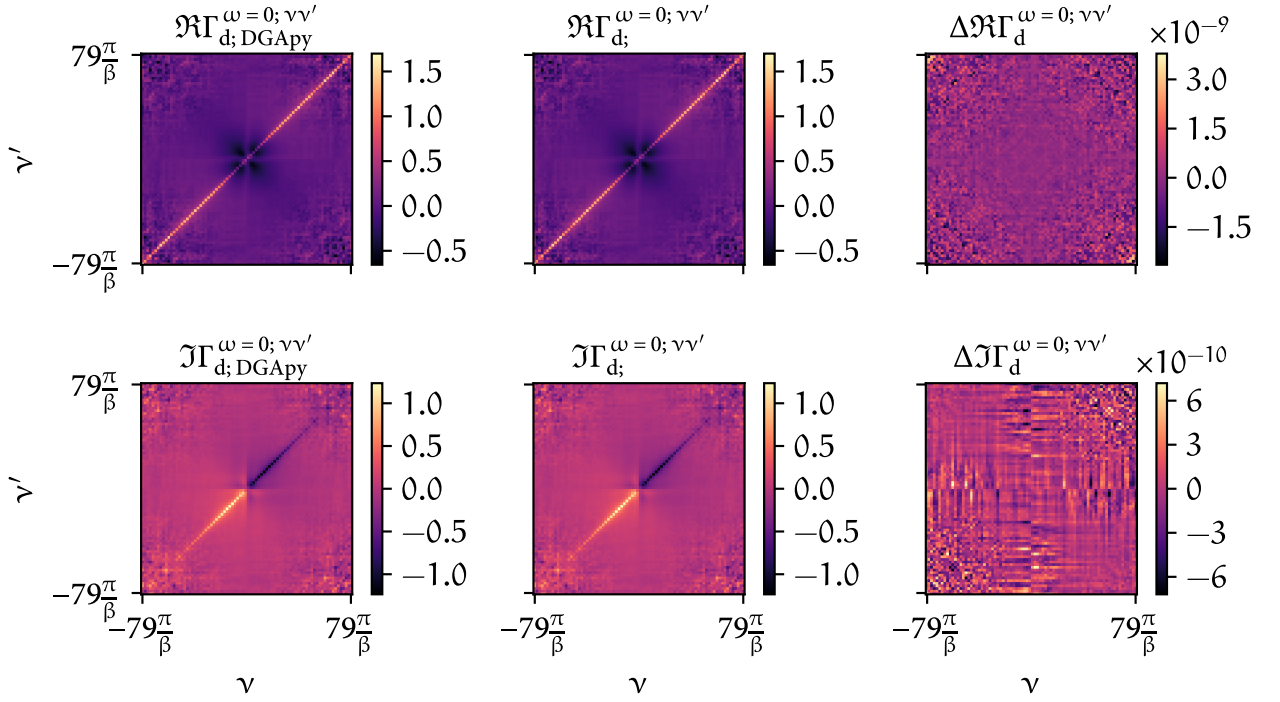


FIGURE 1.3 – Real and imaginary part of the *local* irreducible vertex in the density channel calculated with DGApY (first column) and the new program (second column). Their differences can be seen in the third column.

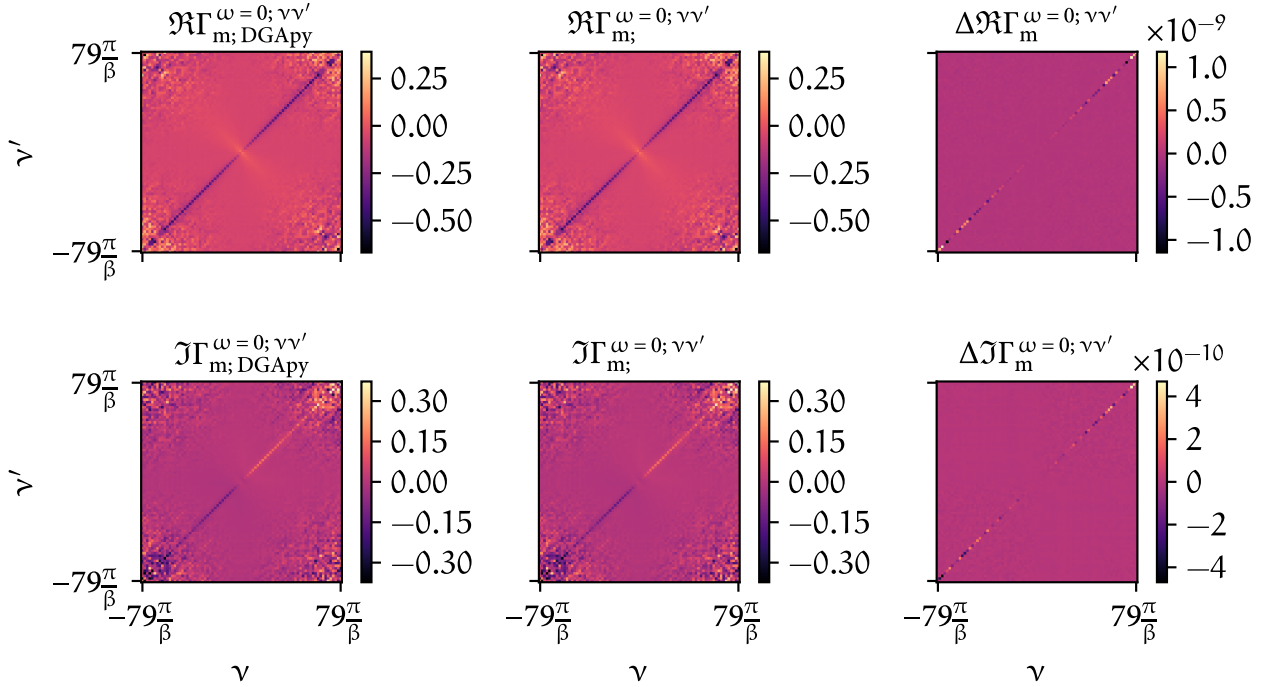


FIGURE 1.4 – Analogous to Fig. 1.3 but for the magnetic spin combination.

1.3.2 Disconnected two-band model

Here, we show successful consistency checks for the multi-orbital implementation of the code using the artificial two-band input discussed earlier. The purpose of these tests was to verify that our code treats such a system in the expected way, namely as two independent orbitals with identical physical properties, and that no artifacts are introduced by the handling of multi-orbital input data. Based on this setup, we carried out three consistency checks:

The first test consisted of projecting a single orbital out of the disconnected two-band dataset generated by `w2dynamics`, perform a single-band $D\Gamma A$ calculation and comparing it to the corresponding orbital of the two-band model. Since both originate from the same underlying data, one expects perfect agreement between the two. Indeed, this is exactly what we observed: both the self-energy and the two-particle vertex functions matched up to numerical precision. This confirmed that our implementation correctly interprets the orbital structure of the disconnected two-band system and does not introduce any discrepancies when picking a specific orbital and compare it to the single-band results. This test case reproduced the the same numerical results (up to floating point precision) and will not be shown here explicitly.

As a second test, we carried out the inverse procedure. Starting from the single-band dataset, we artificially embedded it into a disconnected two-band structure and then compared the $D\Gamma A$ results of one of the diagonal orbitals from this construction to the original single-band data calculated with the program. Again, this comparison did not involve two independent `w2dynamics` calculations, but rather served as a stringent internal consistency check on the code's ability to handle single- and multi-band inputs in a uniform manner. This time our input data had less noise since we only had to perform single-orbital calculations in `w2dynamics`. This also helped to disentangle the results for the third test, where we use different `w2dynamics` calculation runs to check whether differences arise from the DMFT input or the implementation. As expected, the agreement was again exact up to numerical precision for both the self-energy and the vertex functions, confirming that the code faithfully reproduces the equivalence between the single-band description and its trivial two-band embedding. Since this test case reproduced the exact numerical results as expected, we will not show any data similar to above.

The third test, in contrast, did involve two independent input datasets. Here, we used `w2dynamics` to generate data for both the single-band and the disconnected two-band systems separately and then compared the results of the separate $D\Gamma A$ calculations. The results of both datasets agreed very well overall, demonstrating that the expected equivalence between the two cases is also maintained when the input is generated independently. In this comparison, small deviations were visible, however, this behavior is fully consistent with the stochastic nature of the continuous-time quantum Monte Carlo method employed in `w2dynamics`, which is also supported by the first two consistency checks and the respective results showing differences for a variable number of QMC measurements. Statistical noise is an unavoidable feature of CT-QMC data and its presence here is therefore not surprising. Importantly, these fluctuations did not obscure the overall agreement between the single- and two-band results. A comparison between the single- and two-band self-energies and local irreducible vertices for lower QMC measurement statistics can be seen in Fig. 1.5, Fig. 1.6 and Fig. 1.7, respectively. To briefly illustrate the differences that are coming from the `w2dynamics` calculations, we also compared those quantities for a four-times increase of the QMC measurement number in Fig. 1.8, Fig. 1.9 and Fig. 1.10, respectively. This calculation was done for the artificial

two-band model with a $\mathbf{k}$ -grid of $24 \times 24 \times 1$, $n_\omega = 60$, $n_{v;\text{core}} = 40$ and $n_{v;\text{asympt}} = 0$. For the artificial two-band model, we chose to show the (11) -component of the self-energy and the (1111) -component of the irreducible vertex for the comparison. The other orbital-diagonal entry is — except for small stochastic noise — indistinguishable to the first one.

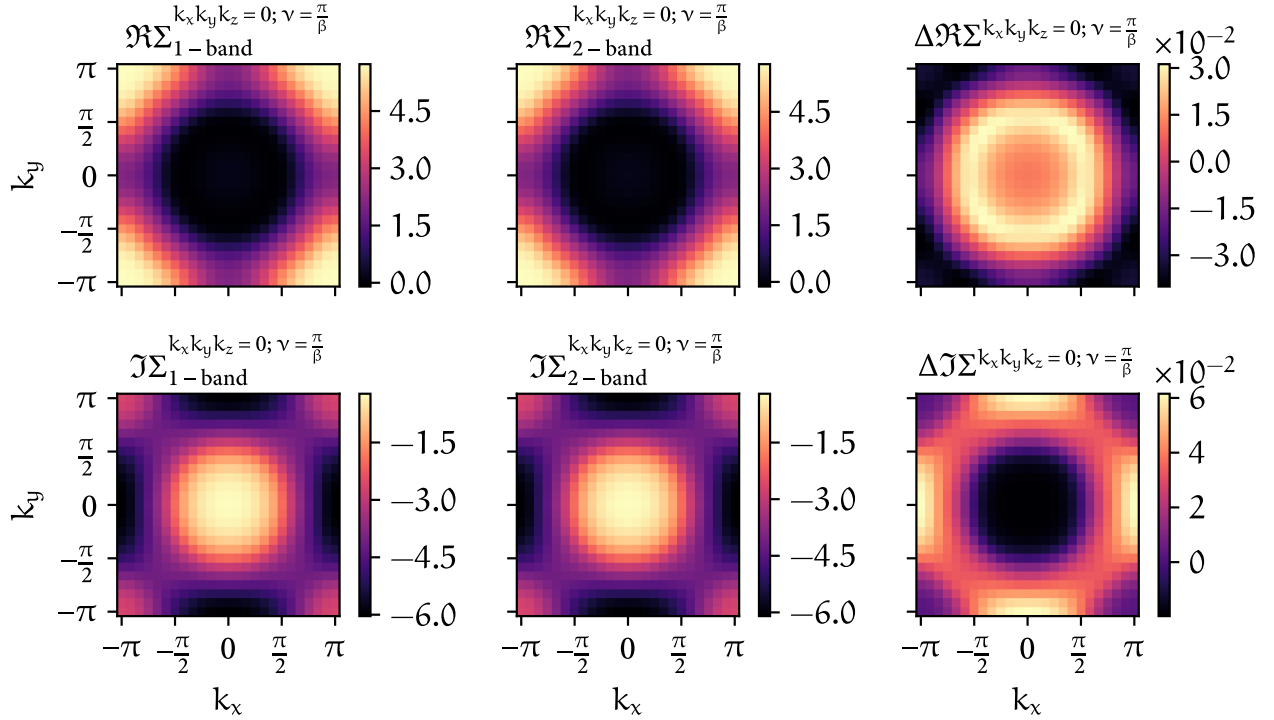


FIGURE 1.5 – One-band self-energy (first column) and the first diagonal component of the self-energy calculated from the artificial two-band system (second column). The difference between both can be seen in the last column and the rows indicate the real- and imaginary part of all self-energies, respectively.

Finally, we also compare the frequency dependence of the one- and two-band self-energy. For this, we plotted the single-band self-energy compared to the first diagonal (11) component of the artificial two-band calculation for both lower (Fig. 1.11(A)) and higher (Fig. 1.11(B)) QMC measurement statistics and $\mathbf{k} = \mathbf{0}$.

Taken together, these three tests confirm that our implementation treats disconnected multi-band systems in a consistent manner. The first two tests established internal consistency within a single dataset by verifying the equivalence of single- and two-band representations derived from the same calculation. The third test then demonstrated that this equivalence is preserved even when comparing truly independent datasets generated by w2dynamics. The results provide strong evidence that our code robustly respects the expected physics of disconnected orbital systems and correctly reproduces the equivalence of two independent copies of the single-band model.

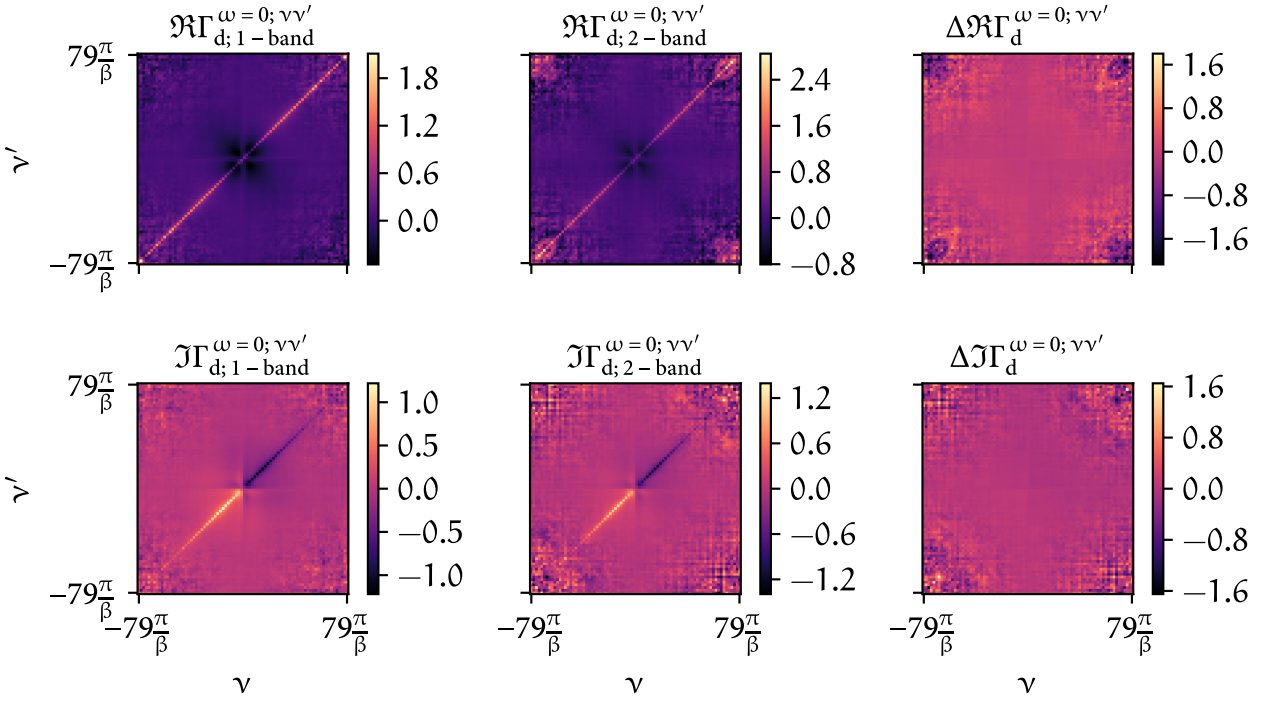


FIGURE 1.6 – Real and imaginary part of the *local* irreducible vertex in the density channel calculated from the one-band dataset (first column) and the artificial two-band dataset (second column). Their differences are shown in the third column.

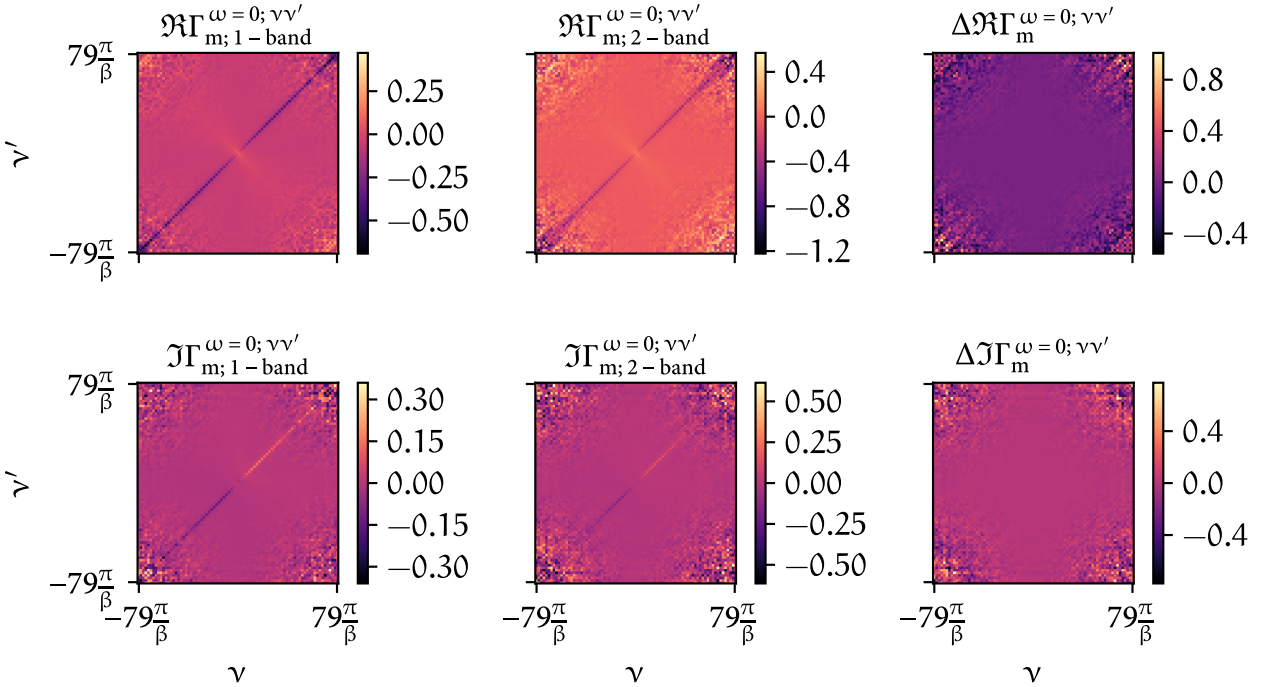


FIGURE 1.7 – Analogous to Fig. 1.6 but for the magnetic spin combination.

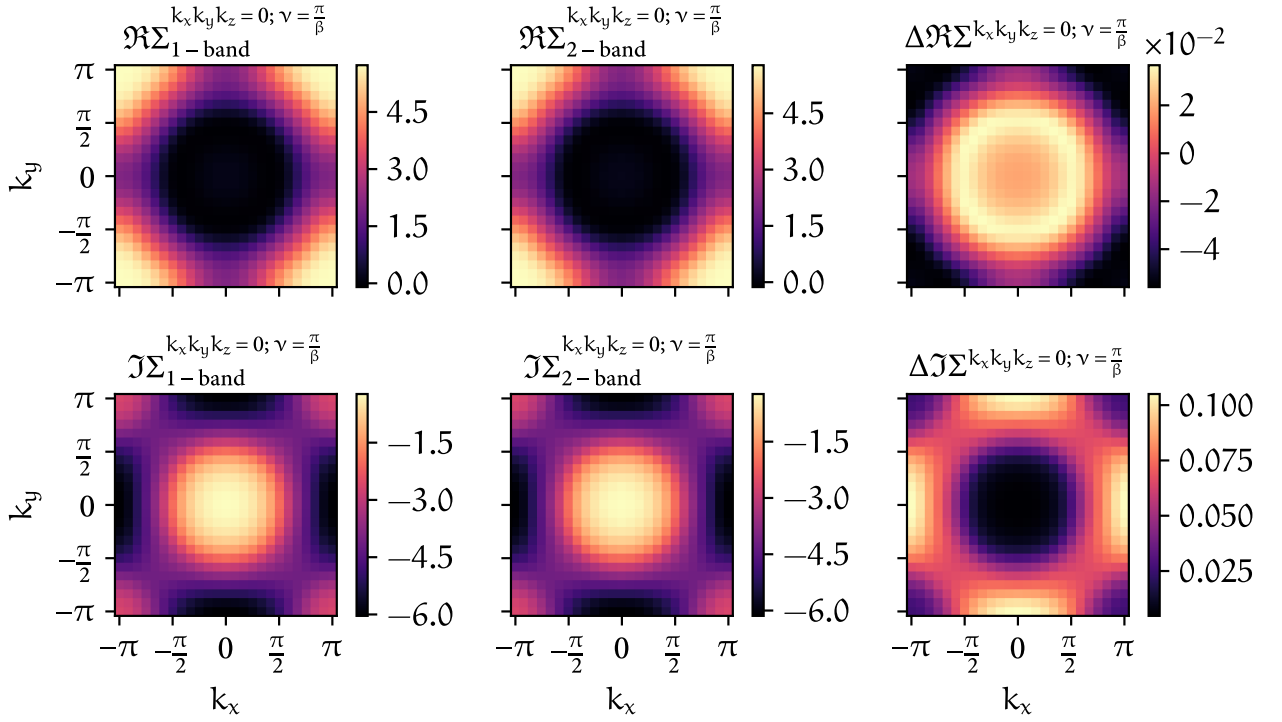


FIGURE 1.8 – Similar to Fig. 1.5 but this time the number of measurement statistics for QMC for the one- and two-particle calculations in w2dynamics were increased by a factor of four.

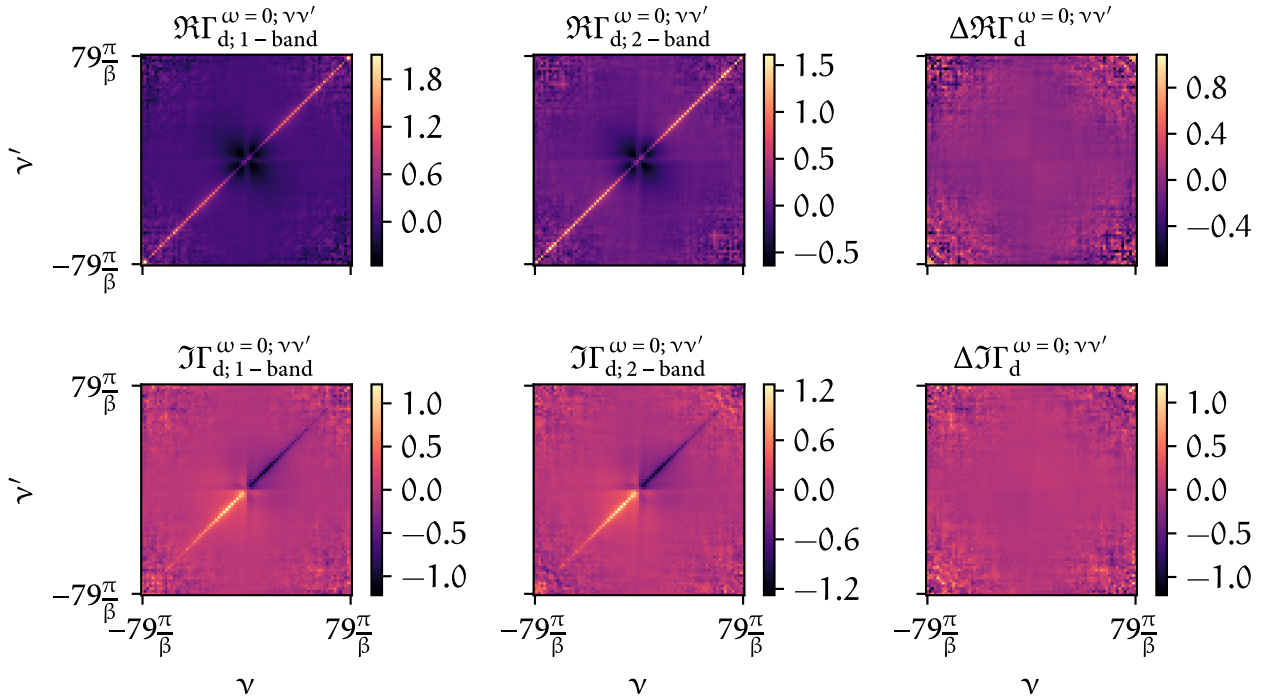


FIGURE 1.9 – Similar to Fig. 1.6 but this time the number of measurement statistics for QMC for the one- and two-particle calculations in w2dynamics were increased by a factor of four.

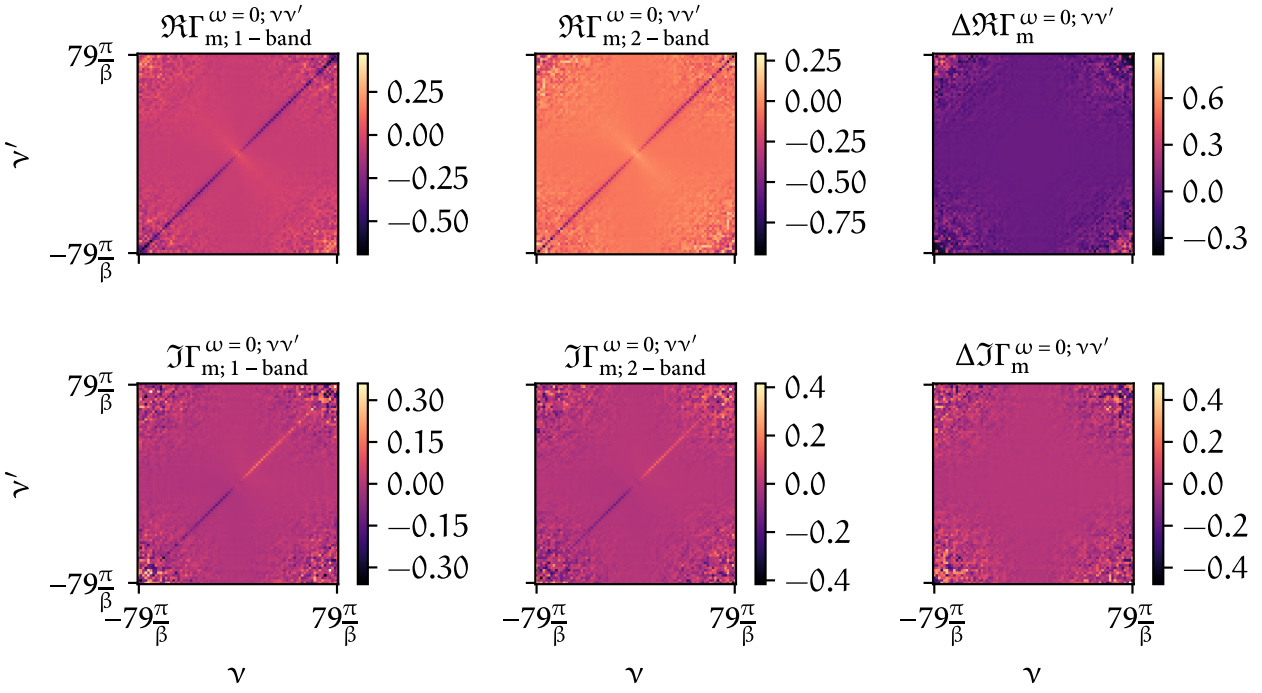


FIGURE 1.10 – Similar to Fig. 1.7 but this time the number of measurement statistics for QMC for the one- and two-particle calculations in w2dynamics were increased by a factor of four.

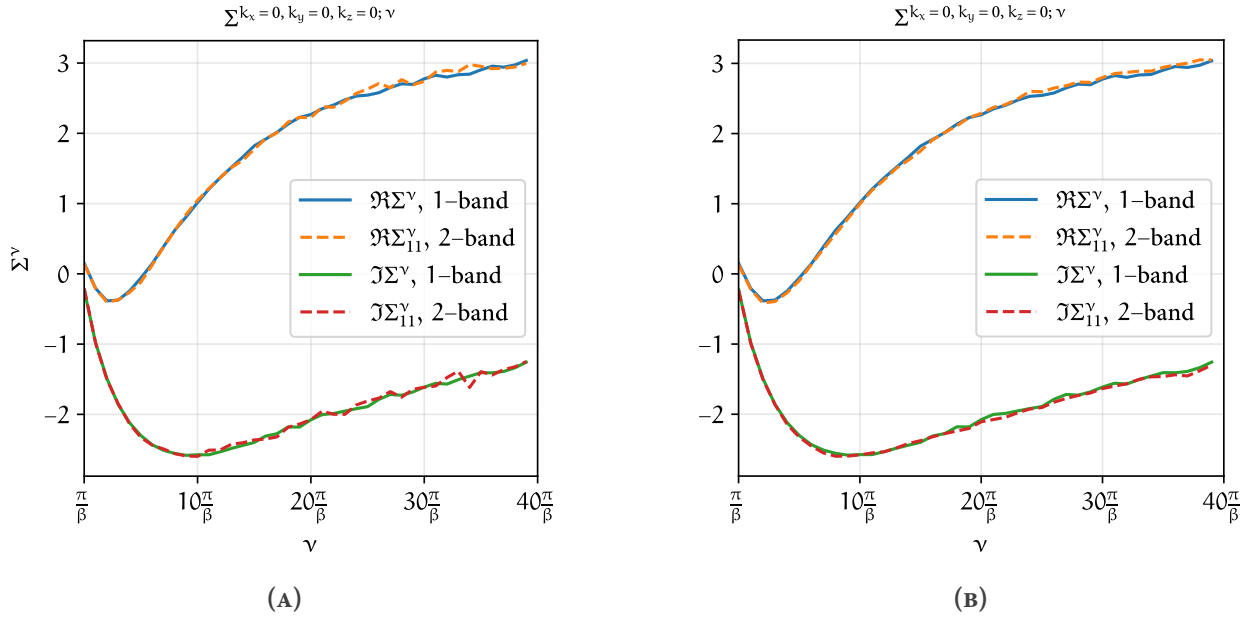


FIGURE 1.11 – Frequency dependence of the real and imaginary part of the self-energy for the single-band and the artificial two-band dataset. (A) shows the comparison for a lower QMC measurement statistics run for the two-band dataset, whereas (B) for a four-times higher number of QMC measurements. The improved behaviour for large frequencies is not surprising, as the increase of QMC measurements mitigates noise.

1.3.3 Rotated disconnected two-band model

For this test, we use the artificial disconnected two-band model introduced earlier and applied rotations in orbital space. The goal is to verify that the orbitals remain properly separated and are not mixed in unintended ways if we rotate the results back at the end, i.e. perform a basis transformation. To this end, we implemented simple rotation routines in the code, which allow us to rotate the one- and two-particle vertex functions in orbital space. For the one-particle Green's function and self-energy this looks like

$$G_{12;\sigma}^{k;\vartheta} = \sum_{ab} (S_{a1}^{\vartheta})^{\dagger} G_{ab;\sigma}^k S_{b2}^{\vartheta}, \quad (1.4)$$

and similarly for $\Sigma_{12;\sigma}^{k;\vartheta}$, whereas for any four-point vertex the rotation is performed like

$$F_{1234;\sigma\sigma'}^{qkk';\vartheta} = \sum_{abcd} (S_{a1}^{\vartheta})^{\dagger} (S_{c3}^{\vartheta})^{\dagger} F_{abcd;\sigma\sigma'}^{qkk'} S_{b2}^{\vartheta} S_{d4}^{\vartheta}. \quad (1.5)$$

Here, the indices $\{i\}$ are representing orbital indices only. Under the hood, this transforms the creation and annihilation operators of the objects with S being a two-dimensional rotation matrix, which simply looks like

$$S_{12}^{\vartheta} = \begin{pmatrix} \cos(\vartheta) & \sin(\vartheta) \\ -\sin(\vartheta) & \cos(\vartheta) \end{pmatrix}_{12} \quad \text{and} \quad (S_{12}^{\vartheta})^{\dagger} = S_{21}^{\vartheta} = S_{12}^{-\vartheta}. \quad (1.6)$$

To perform this test, we applied a rotation by an angle ϑ to all input quantities from DMFT and the Hamiltonian. The D Γ A algorithm was then used to compute the non-local self-energy, which we subsequently rotated back by $-\vartheta$. We compared the resulting self-energy to a reference calculation performed without any rotations and found the individual elements to agree up to floating-point precision, with $\sum_{k;12} |\Sigma_{12}^{k;\vartheta \neq 0} - \Sigma_{12}^{k;\vartheta=0}| \lesssim 10^{-9}$ for $\vartheta = \frac{\pi}{2}, \pi$ and $\sum_{k;12} |\Sigma_{12}^{k;\vartheta \neq 0} - \Sigma_{12}^{k;\vartheta=0}| \lesssim 10^{-3}$ for other values of ϑ . We tried this with different numbers of Matsubara frequencies n_{ω} and n_{ν} and varying k -grid sizes — all yielding indistinguishable results.

Exemplary results of the self-energy can be seen in Fig. 1.12 and for the irreducible vertex $\Gamma_{d/m}^{\omega=0;\nu\nu'}$ in Fig. 1.13 and Fig. 1.14. This calculation was done for the artificial two-band model with a k -grid of $24 \times 24 \times 1$, $n_{\omega} = 60$, $n_{\nu;\text{core}} = 40$ and $n_{\nu;\text{asympt}} = 0$. Here, we show the (11) orbital of the self-energy and the (1111) orbital component of the irreducible vertex; the (22) and (2222) components of both quantities look very similar.

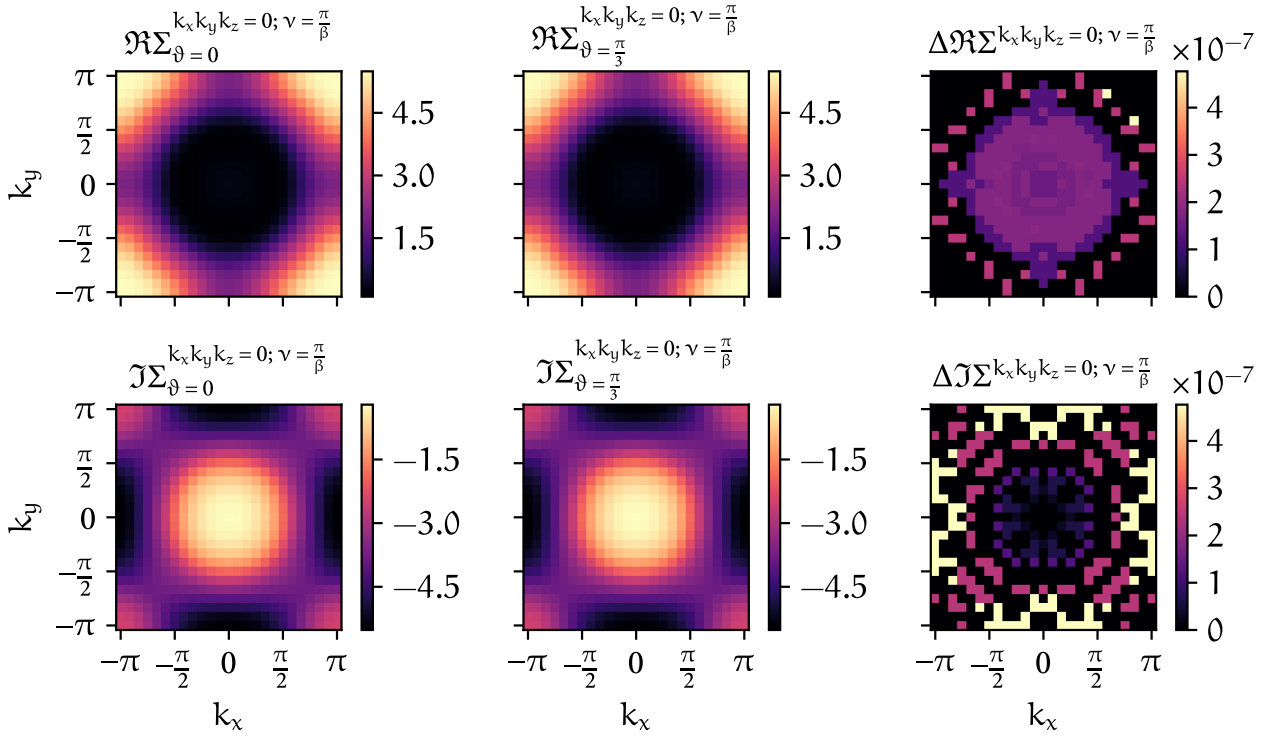


FIGURE 1.12 – Two-band calculation without a rotation ($\vartheta = 0$, first column) and a calculation, where the input data was rotated before the D Γ A calculation and subsequently rotated back by a factor of $\vartheta = \frac{\pi}{3}$ (second column) for both the real and imaginary part of the self-energy (first and second row, respectively). Their differences are shown in the third column.

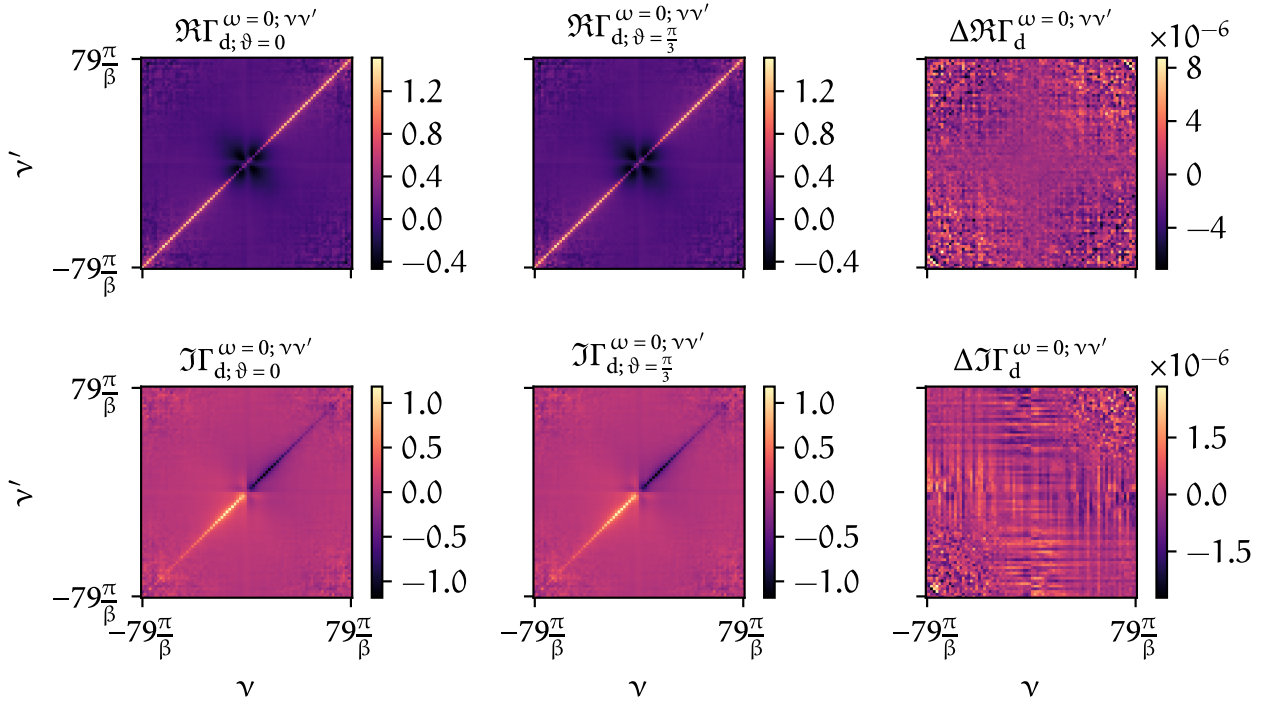


FIGURE 1.13 – Two-band calculation without a rotation ($\vartheta = 0$, first column) and a calculation, where the input data was rotated before the $D\Gamma A$ calculation and subsequently rotated back by a factor of $\vartheta = \frac{\pi}{3}$ (second column) for both the real and imaginary part of the *local* irreducible vertex $\Gamma_d^{\omega=0; \nu\nu'}$ (first and second row, respectively) in the density spin combination. . Their differences are shown in the third column.

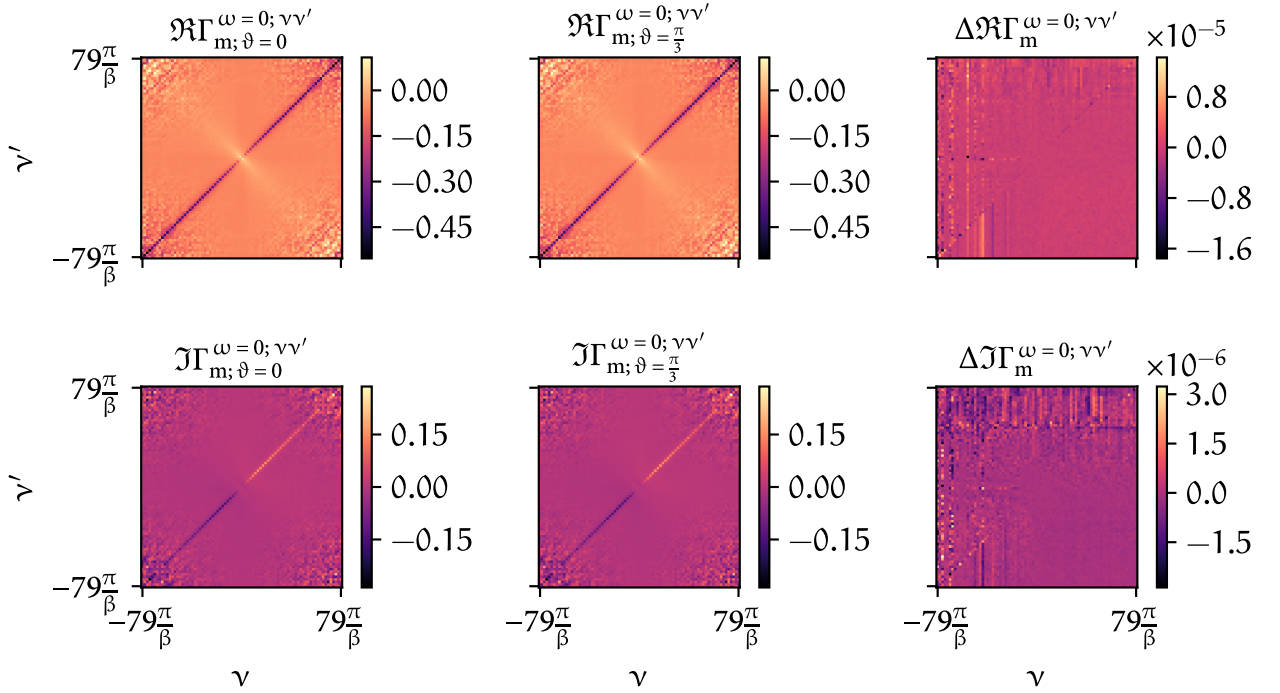


FIGURE 1.14 – Analogous to Fig. 1.13 but for the magnetic spin combination.